リスナー一覧を取得

今部屋にいるリスナーの `id` とニックネームを取得します。抽選や出席確認のように「今誰がいるか」が必要なときに使います。

メソッド・パス	`GET /v1/live/listeners`
必要な権限	`listeners.read`
成功レスポンス	`200`
配信中でないとき	`404`（`OAPI_MNGR_0301`）

人気の配信はリスナーが数千人になるため、一度にすべては返さず **カーソルで続きを受け取ります。**

```
export SPOON_BASE_URL="https://jp-openapi.spooncast.net"
export SPOON_ACCESS_TOKEN="{発行された access token}"
```

韓国リージョンなら `https://kr-openapi.spooncast.net` です — リージョン別アドレス。

リクエスト

```
GET /v1/live/listeners?cursor={cursor}
```

ヘッダー

名前	必須	説明
`Authorization`	✔	`Bearer {access_token}` 形式です。

クエリパラメータ

名前	型	必須	説明
`cursor`	string		前のレスポンスの `nextCursor` をそのまま入れます。**最初のページでは省略します。**このボット・この DJ 連携で受け取った値だけ使えます。

リクエスト例

```
# 最初のページ
curl -H "Authorization: Bearer $SPOON_ACCESS_TOKEN"
"$SPOON_BASE_URL/v1/live/listeners"

# 次のページ — 前のレスポンスの nextCursor をそのまま入れます
curl -H "Authorization: Bearer $SPOON_ACCESS_TOKEN" \
  "$SPOON_BASE_URL/v1/live/listeners?cursor=Qm9vTXp4c1ZhYk5yX2xPcQ"
```

レスポンス

```
{
  "listeners": [
    { "id": "Ab1Cd2Ef3Gh4Ij5Kl6Mn7O", "nickname": "スプーンリスナー" },
    { "id": "Pq8Rs9Tu0Vw1Xy2Za3Bc4D", "nickname": "ラジオファン" }
  ],
  "nextCursor": "Qm9vTXp4c1ZhYk5yX2xPcQ"
}
```

フィールド	型	null	説明
`listeners`	object[]	いいえ	今部屋にいるリスナーです。
`listeners[].id`	string	いいえ	リスナーを指す値です。暗号化した値です。
`listeners[].nickname`	string	いいえ	ニックネームです。

フィールド	型	null	説明
`nextCursor`	string	はい	次のページのカーソルです。`null` なら最後のページです。

> **`id` で人を識別します**
>
> `id` はその人の一意な識別子です。同じ人なら常に同じ値なので、出席・累計・ランキングのキーとして 使います。`nickname` は変わり、重複するので識別に使わないでください。
>
> 値の範囲は自分のボットとその DJ の組み合わせです。DJ が連携を解除して再度同意しても維持されます。

この `id` は **イベントストリームの `user.id` と同じ値** です。チャットを送った人が今も部屋に いるかを照合できます。

コード

```
// listeners.ts — node --experimental-strip-types listeners.ts

interface Listener {
  id: string;
  nickname: string;
}

interface ListenersPage {
  listeners: Listener[];
  nextCursor: string | null;
}

const BASE = process.env.SPOON_BASE_URL;
const TOKEN = process.env.SPOON_ACCESS_TOKEN;

const all = [];
let cursor;

do {
  const url = new URL(`${BASE}/v1/live/listeners`);
  if (cursor) url.searchParams.set('cursor', cursor);
```

```
  const res = await fetch(url, { headers: { Authorization: `Bearer ${TOKEN}` } });
  if (!res.ok) throw new Error(`リスナー一覧の取得に失敗しました: ${res.status}`);

  const page = (await res.json()) as ListenersPage;
  all.push(...page.listeners);
  cursor = page.nextCursor;    // null なら繰り返しが終わります
} while (cursor);

console.log(`今 ${all.length}人が聴いています。`);

// 一人を選びます – 保存・照合のキーは常に id です
const winner = all[Math.floor(Math.random() * all.length)];
console.log(`抽選: ${winner.nickname} (${winner.id})`);
```

```
node --experimental-strip-types listeners.ts
```

注意

ボット自身は一覧に出ません。イベントストリームと同じルールです。そのため 1 ページに 届く件数は毎回同じとは限りません。最後のページかどうかは件数ではなく、`nextCursor` が `null` かどうかで判断します。

`nextCursor` が `null` になるまで繰り返します。最初のページだけで終えると、リスナーが 多い配信では一部しか見えません。逆に `null` をそのまま `?cursor=` に入れてはいけません — 繰り返しを止める合図です。

一覧は **呼んだ瞬間のスナップショット** です。ページをめくる間に人が出入りします。「配信中ずっといたか」が必要なら イベントストリームの入場イベントを受け取り、自分で積み上げます。

エラー

HTTP	`detailCode`	何が起きたか	どうするか
`400`	`OAPI_MNGR_0103`	`cursor` が正しくありません。前のレスポンスの `nextCursor` ではないか、別のボット・別の DJ 連携で受け取った値です。	`cursor` なしで最初のページからやり直します。

HTTP	`detailCode`	何が起きたか	どうするか
`401`	なし	トークンがないか無効です。	DJ に再連携を案内します。
`403`	なし	`listeners.read` の権限がありません。	権限を追加して再同意を得ます。
`404`	`OAPI_MNGR_0301`	配信中ではありません。	エラーではなく状態です。しばらくして呼び直します。
`502`	`OAPI_MNGR_2302`	配信サーバーのエラーです。	バックオフしてから再試行します。

この表にない状態も返ることがあります。 誤ったメソッド・`Content-Type`(`400`・`405`・`415`、`OAPI_MNGR_0103`)・呼び出し上限(`429`、ボディなし)・サーバーエラー(`500`、`OAPI_MNGR_1002`)は **すべてのエンドポイントに共通** です — エラーコード にあります。

関連ドキュメント

- エンドポイント一覧 — エンドポイントの全一覧です。
- ファンランキングを取得 — ギフトが多い順の上位 30 名です。
- イベントストリーム — 同じ `id` でチャット・入場を受け取ります。
- エラーコード — エラーコードの一覧です。